Appendix

A Convergence of Minibatch HSIC

Proposition 1. *Let $\boldsymbol{K} \in \mathbb{R}^{m\times m}$ and $\boldsymbol{L} \in \mathbb{R}^{m\times m}$ be two kernel matrices constructed by applying kernel functions k and l respectively to all pairs of examples in a dataset $\mathcal{D}$. Form c random partitionings p of $\mathcal{D}$ into m/n minibatches b of size n, and let $\tilde{\boldsymbol{K}}^{b,p} \in \mathbb{R}^{n\times n}$ and $\tilde{\boldsymbol{L}}^{b,p} \in \mathbb{R}^{n\times n}$ be kernel matrices constructed by applying kernels k and l to all pairs of examples within each minibatch. Define $U_0 = \mathrm{HSIC}_1(\boldsymbol{K}, \boldsymbol{L})$, the value of HSIC_1 applied to the full dataset, and $\tilde{U}_p = \frac{n}{m}\sum_{b=1}^{m/n} \mathrm{HSIC}_1(\boldsymbol{K}^{b,p}, \boldsymbol{L}^{b,p})$, the average value of HSIC_1 over the minibatches in partitioning (epoch) p. Then $\frac{1}{c}\sum_{p=1}^{c} \tilde{U}_p \xrightarrow{P} U_0$ as $c \to \infty$.*

Proof. Let $\boldsymbol{i}_4^m$ be the set of all 4-tuples of indices between 1 and m where each index occurs exactly once. As proven in Theorem 3 of Song et al. (2012), U_0 is a U-statistic:

$$U_0 = \mathrm{HSIC}_1(\boldsymbol{K}, \boldsymbol{L}) = \frac{(m-4)!}{m!} \sum_{S\in \boldsymbol{i}_4^m} h(K_S, L_S), \tag{5}$$

where $K_{(i,j,q,r)} = (K_{i,j}, K_{i,q}, K_{i,r}, K_{j,q}, K_{j,r}, K_{q,r})$ and the kernel of the U-statistic h is defined in Song et al. (2012). Let δ_S^b be 1 if the 4-tuple of dataset indices S is selected in minibatch b and 0 otherwise. Then:

$$\tilde{U}_p = \frac{(n-4)!}{n!}\frac{n}{m}\sum_{b=1}^{m/n}\sum_{S\in \boldsymbol{i}_4^m} \delta_S^b h(K_S, L_S). \tag{6}$$

Taking the expectation with respect to $\boldsymbol{\delta}$, and noting that $\boldsymbol{\delta}$ is independent of $h(K_S, L_S)$,

$$\mathbb{E}_{\boldsymbol{\delta}}[\tilde{U}_p] = \frac{(n-4)!}{n!}\frac{n}{m}\sum_{b=1}^{m/n}\sum_{S\in \boldsymbol{i}_4^m} \mathbb{E}_{\boldsymbol{\delta}}\left[\delta_S^b h(K_S, L_S)\right] \tag{7}$$

$$= \frac{(n-4)!}{n!}\frac{n}{m}\sum_{b=1}^{m/n}\sum_{S\in \boldsymbol{i}_4^m} \mathbb{E}_{\boldsymbol{\delta}}\left[\delta_S^b\right] h(K_S, L_S). \tag{8}$$

By symmetry, $\mathbb{E}_{\boldsymbol{\delta}}\left[\delta_S^b\right]$ is the same for all example and batch indices. Specifically, there are $n!/(n-4)!$ 4-tuples that can be formed from each batch and $m!/(m-4)!$ 4-tuples that can be formed from the entire dataset, so the probability that a given 4-tuple is in a given batch is $\mathbb{E}_{\boldsymbol{\delta}}\left[\delta_S^b\right] = (n!/(n-4)!)/(m!/(m-4)!)$. Thus:

$$\mathbb{E}_{\boldsymbol{\delta}}[\tilde{U}_p] = \frac{(m-4)!}{m!}\sum_{S\in \boldsymbol{i}_4^m} h(K_S, L_S) = U_0. \tag{9}$$

The minibatch indicators δ_S^b are either 0 or 1, so their variances and covariances are bounded, and the weighted sum in Eq. 6 has finite variance. Thus, by the law of large numbers, $\frac{1}{c}\sum_{p=1}^{c} \tilde{U}_p \xrightarrow{P} U_0$ as $p \to \infty$. □

B Training Details

Our CIFAR-10 and CIFAR-100 networks follow the same architecture as He et al. (2016); Zagoruyko & Komodakis (2016). We train a set of models where we fix the width multiplier of deep networks to 1 and experiment with models of depths 32, 44, 56, 110, 164. On CIFAR-100, the block structure only appears at a greater depth so we also include depths 218 and 224 in our investigation. For wide networks, we examine width multipliers of 1, 2, 4, 8 and 10 and depths of 14, 20, 26, and 38. We use SGD with momentum of 0.9, together with a cosine decay learning rate schedule and batch size of 128, to train each model for 300 epochs. Models are trained with standard CIFAR-10 data augmentation comprising random flips and translations of up to 4 pixels.

Each depth and width configuration is trained with 10 different seeds for CKA analysis, and 200 seeds for model predictions comparison.

On ImageNet, we start with the ResNet-50 architecture and increase depth or width in the third stage only, following the scaling approach of (He et al., 2016). We train for 120 epochs using SGD with momentum of 0.9 and a cosine decay learning rate schedule at a batch size of 256. We use 100 seeds for model prediction comparison.

For experiments with reduced dataset size, we subsample the training data from the original CIFAR training set by the corresponding proportion, keeping the number of samples for each class the same. All CKA results are then computed based on the full CIFAR test set.

Table B.1: Accuracy of examined neural networks on CIFAR-10 and CIFAR-100.

Depth	Width	CIFAR-10 Test Accuracy (%)	CIFAR-100 Test Accuracy (%)
32	1	93.5	71.2
44	1	94.0	72.0
56	1	94.2	73.3
110	1	94.3	74.0
164	1	94.4	73.9
14	1	92.0	67.8
14	2	94.1	72.9
14	4	95.4	77.0
14	8	95.9	80.0
14	10	96.0	80.2
20	1	92.8	69.4
20	2	94.6	74.4
20	4	95.4	77.6
20	8	96.0	80.2
20	10	95.8	80.8
26	1	93.3	70.5
26	2	94.9	75.8
26	4	95.6	79.3
26	8	95.9	80.9
26	10	95.8	81.0
38	1	93.8	72.3
38	2	95.1	75.9
38	4	95.5	78.6
38	8	95.7	79.8
38	10	95.7	80.5

C Block Structure in a Different Architecture

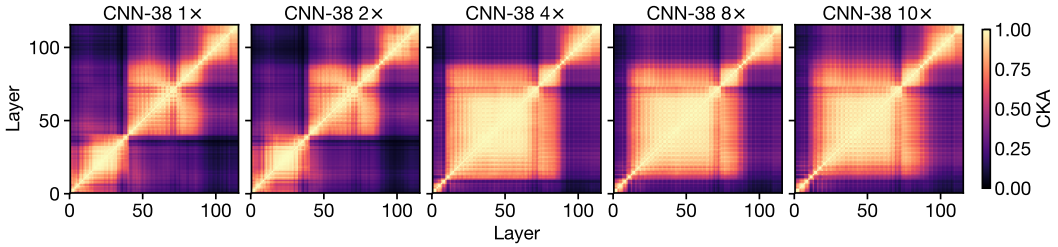

Figure C.1: Block structure also appears in models without residual connections. We remove residual connections from existing CIFAR-10 ResNets and plot CKA heatmaps for layers in the resulting architecture after training. Since the lack of residual connections prevents deep networks from performing well on the task, here we only show the representational similarity for models of increasing width. As previously observed in Figure 1, the block structure emerges in higher capacity models.

D Probing the Block Structure

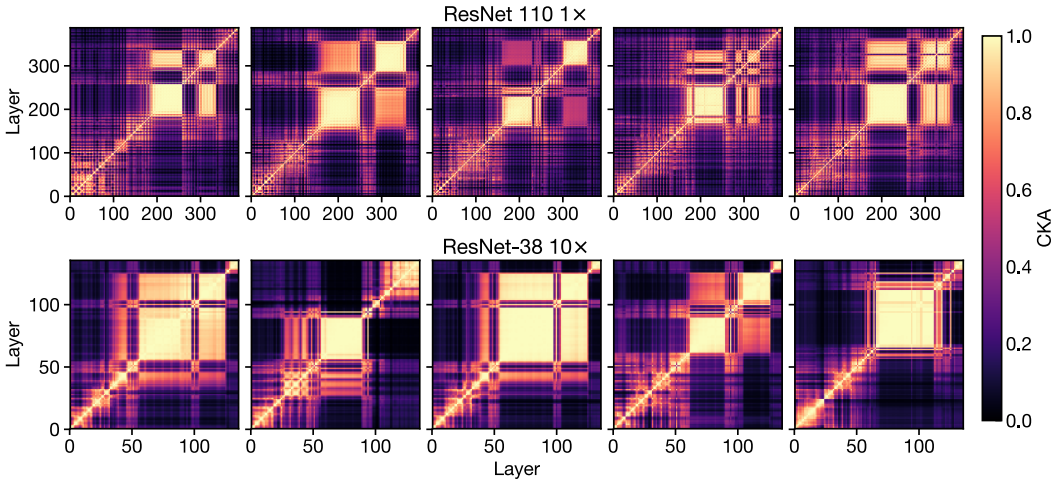

Figure D.1: Block structure varies across random initializations. We plot CKA heatmaps as in Figure 1 for 5 random seeds of a deep model (top row) and a wide model (bottom row) trained on CIFAR-10. While the size and position vary, the block structure is clearly visible in all seeds.